

DSA Master Roadmap

From Zero to Problem Solver

20 years of industry wisdom · Patterns-first approach · 5 Phases · 13 Topics · 75+ Problems

Instructor's Core Philosophy: Don't memorise code. Learn to recognise the SIGNAL — the problem clue that tells you which pattern to reach for. Videos give you recognition, not recall. Always: Watch concept → Learn 1 pattern → Solve 2–3 problems of that pattern → Then move on.

ROADMAP OVERVIEW

Phase	Title	Duration	Topics
Phase 1	Foundation	4–5 weeks	3 topics
Phase 2	Linear Structures	3–4 weeks	2 topics
Phase 3	Trees & Graphs	5–6 weeks	3 topics
Phase 4	Dynamic Programming	5–6 weeks	2 topics
Phase 5	Advanced Topics	3–4 weeks	3 topics

Phase 1: Foundation

4–5 weeks

■ Arrays & Strings

THEORY — LEARN FIRST

- Memory layout — contiguous allocation
- Indexing, slicing, iteration
- Mutability vs immutability
- 2D arrays, jagged arrays

PATTERNS

Two Pointers

Left & right pointers converging. Used when array is sorted or you need pair-based comparisons.

SIGNAL: Sorted array + pair / triplet / subarray sum target

Sliding Window

Expand/shrink a window of elements. Used for contiguous subarray/substring problems.

SIGNAL: 'Max/min subarray of size k' or 'longest substring with condition'

Prefix Sum

Precompute cumulative sums for $O(1)$ range queries.

SIGNAL: Range sum queries, subarray sum equals k

Kadane's Algorithm

Track running max subarray sum.

SIGNAL: Maximum subarray sum

PROBLEMS — SOLVE IN ORDER

#	Problem	Pattern	Difficulty
1	Two Sum	Hashing	Easy
2	Best Time to Buy & Sell Stock	Kadane's	Easy
3	Contains Duplicate	Hashing	Easy
4	Maximum Subarray	Kadane's	Medium
5	Product of Array Except Self	Prefix Sum	Medium

6	3Sum	Two Pointers	Medium
7	Container With Most Water	Two Pointers	Medium
8	Longest Substring Without Repeating	Sliding Window	Medium
9	Subarray Sum Equals K	Prefix Sum + Hash	Medium
10	Trapping Rain Water	Two Pointers / Prefix	Hard

Hashing

THEORY — LEARN FIRST

- Hash functions, collision resolution
- HashMap vs HashSet
- Load factor, rehashing
- Frequency counting

PATTERNS

Frequency Map

Count occurrences to find duplicates, majorities, or anagrams.

SIGNAL: 'Find duplicate', 'anagram check', 'majority element'

Complement Lookup

Store seen values; for each new value check if its complement exists.

SIGNAL: Two-sum style — find pair with target

Running State Hash

Hash prefix states (sums, XORs) to detect subarrays with a property.

SIGNAL: 'Subarray with sum 0', 'equal 0s and 1s'

PROBLEMS — SOLVE IN ORDER

#	Problem	Pattern	Difficulty
1	Valid Anagram	Frequency Map	Easy
2	Group Anagrams	Frequency Map	Medium
3	Top K Frequent Elements	Freq Map + Bucket Sort	Medium
4	Longest Consecutive Sequence	HashSet	Medium
5	Subarray with Sum 0	Prefix Sum + Hash	Medium

■ Recursion & Backtracking

THEORY — LEARN FIRST

- Call stack intuition
- Base case design
- Tree of recursive calls
- Memoization intro

PATTERNS

Choose / Explore / Unchoose

Classic backtracking template — make a choice, recurse, undo choice.

SIGNAL: Generate all subsets, permutations, combinations

Pruning

Skip branches that can't lead to valid solutions early.

SIGNAL: Sudoku solver, N-Queens, word search

PROBLEMS — SOLVE IN ORDER

#	Problem	Pattern	Difficulty
1	Subsets	Backtracking	Medium
2	Permutations	Backtracking	Medium
3	Combination Sum	Backtracking + Pruning	Medium
4	Word Search	DFS + Backtracking	Medium
5	N-Queens	Backtracking + Pruning	Hard
6	Palindrome Partitioning	Backtracking	Medium

Phase 2: Linear Structures

3–4 weeks

■ Linked Lists

THEORY — LEARN FIRST

- Singly vs Doubly vs Circular
- Pointer manipulation
- Dummy node trick
- In-place reversal

PATTERNS

Fast & Slow Pointers

Two pointers at different speeds to find cycles, midpoints.

SIGNAL: Cycle detection, find middle, Nth from end

In-Place Reversal

Reverse links iteratively using prev/curr/next.

SIGNAL: Reverse list, reverse sublist, palindrome check

Merge / Two-List

Use two pointers to merge sorted lists.

SIGNAL: Merge two sorted lists, merge K lists

PROBLEMS — SOLVE IN ORDER

#	Problem	Pattern	Difficulty
1	Reverse Linked List	In-Place Reversal	Easy
2	Linked List Cycle	Fast & Slow	Easy
3	Middle of Linked List	Fast & Slow	Easy
4	Merge Two Sorted Lists	Two Pointers	Easy
5	Remove Nth Node From End	Fast & Slow	Medium
6	Reorder List	Fast & Slow + Reversal	Medium
7	LRU Cache	HashMap + DLL	Medium
8	Merge K Sorted Lists	Heap / Divide & Conquer	Hard

■ Stacks & Queues

THEORY — LEARN FIRST

- LIFO vs FIFO
- Monotonic stacks
- Stack using arrays vs linked list
- Deque & its uses

PATTERNS

Monotonic Stack

Maintain increasing or decreasing stack to find next greater/smaller element.

SIGNAL: Next Greater Element, Largest Rectangle, Daily Temperatures

Balanced Parentheses

Push open brackets; on close, pop and match.

SIGNAL: Validate brackets, min add to make valid

BFS with Queue

Level-order traversal of trees/graphs.

SIGNAL: Level-order, shortest path in unweighted graph

PROBLEMS — SOLVE IN ORDER

#	Problem	Pattern	Difficulty
1	Valid Parentheses	Stack	Easy
2	Daily Temperatures	Monotonic Stack	Medium
3	Next Greater Element I	Monotonic Stack	Easy
4	Largest Rectangle in Histogram	Monotonic Stack	Hard
5	Min Stack	Stack Design	Medium
6	Sliding Window Maximum	Monotonic Deque	Hard

Phase 3: Trees & Graphs

5–6 weeks

T Binary Trees

THEORY — LEARN FIRST

- Tree terminology: root, leaf, height, depth
- DFS: preorder, inorder, postorder
- BFS: level-order traversal
- Recursive tree thinking

PATTERNS

DFS on Tree

Recursively process left subtree, node, right subtree.

SIGNAL: Path sum, height, diameter, mirror check

BFS / Level-Order

Queue-based traversal level by level.

SIGNAL: Level averages, right side view, zigzag traversal

Return Value Propagation

Each node computes a local answer and returns it up.

SIGNAL: Diameter, max path sum, balanced check

PROBLEMS — SOLVE IN ORDER

#	Problem	Pattern	Difficulty
1	Maximum Depth of Binary Tree	DFS	Easy
2	Invert Binary Tree	DFS	Easy
3	Symmetric Tree	DFS	Easy
4	Level Order Traversal	BFS	Medium
5	Binary Tree Right Side View	BFS	Medium
6	Diameter of Binary Tree	DFS + Return Value	Easy
7	Path Sum II	DFS + Backtracking	Medium
8	Binary Tree Maximum Path Sum	DFS + Return Value	Hard

B BST

THEORY — LEARN FIRST

- BST invariant: $\text{left} < \text{root} < \text{right}$
- Search, insert, delete in $O(\log n)$
- In-order = sorted sequence
- Balanced vs unbalanced trees

PATTERNS

BST Property Exploit

Use $\text{left} < \text{root} < \text{right}$ to navigate without visiting all nodes.

SIGNAL: Validate BST, find Kth smallest, closest value

In-order Traversal

In-order of BST yields sorted output.

SIGNAL: Convert sorted array to BST, in-order successor

PROBLEMS — SOLVE IN ORDER

#	Problem	Pattern	Difficulty
1	Validate Binary Search Tree	DFS + Bounds	Medium
2	Kth Smallest Element in BST	In-order	Medium
3	Lowest Common Ancestor of BST	BST Property	Medium
4	Convert Sorted Array to BST	Divide & Conquer	Easy

G Graphs

THEORY — LEARN FIRST

- Adjacency list vs matrix
- Directed vs undirected
- Connected components
- Topological sort concept

PATTERNS

DFS on Graph

Mark visited; recursively explore neighbors.

SIGNAL: Island count, connected components, path existence

BFS Shortest Path

BFS guarantees shortest path in unweighted graph.

SIGNAL: Shortest path, word ladder, rotten oranges

Union-Find

Track connected components with path compression.

SIGNAL: Redundant connection, number of provinces

Topological Sort

Order nodes with dependencies (Kahn's or DFS).

SIGNAL: Course schedule, task ordering

PROBLEMS — SOLVE IN ORDER

#	Problem	Pattern	Difficulty
1	Number of Islands	DFS/BFS	Medium
2	Clone Graph	DFS + HashMap	Medium
3	Rotting Oranges	BFS	Medium
4	Course Schedule	Topological Sort	Medium
5	Word Ladder	BFS	Hard
6	Pacific Atlantic Water Flow	DFS from borders	Medium
7	Number of Connected Components	Union-Find / DFS	Medium
8	Alien Dictionary	Topological Sort	Hard

Phase 4: Dynamic Programming

5–6 weeks

D 1D Dynamic Programming

THEORY — LEARN FIRST

- Overlapping subproblems
- Optimal substructure
- Memoization (top-down) vs Tabulation (bottom-up)
- State definition is the hardest and most important part

PATTERNS

Linear DP

$dp[i]$ depends on $dp[i-1]$ or $dp[i-2]$.

SIGNAL: Climbing stairs, house robber, Fibonacci variants

Decision DP

At each index choose to include or skip.

SIGNAL: Coin change, jump game, word break

PROBLEMS — SOLVE IN ORDER

#	Problem	Pattern	Difficulty
1	Climbing Stairs	Linear DP	Easy
2	House Robber	Decision DP	Medium
3	House Robber II	Decision DP	Medium
4	Coin Change	Decision DP	Medium
5	Word Break	Decision DP	Medium
6	Longest Increasing Subsequence	DP + Binary Search	Medium
7	Jump Game II	Greedy / DP	Medium

D 2D Dynamic Programming

THEORY — LEARN FIRST

- State as (i, j) grid or (i, j) string indices
- Unique paths intuition

- String DP: LCS, Edit Distance
- Knapsack variants

PATTERNS

Grid DP

$dp[i][j]$ = best result to reach cell (i,j) .

SIGNAL: Unique paths, min path sum, maximal square

String DP

$dp[i][j]$ comparing prefix of $s1[i]$ with $s2[j]$.

SIGNAL: LCS, edit distance, interleaving strings

0/1 Knapsack

For each item, include or exclude. $dp[i][w]$.

SIGNAL: Partition equal subset, target sum

PROBLEMS — SOLVE IN ORDER

#	Problem	Pattern	Difficulty
1	Unique Paths	Grid DP	Medium
2	Minimum Path Sum	Grid DP	Medium
3	Longest Common Subsequence	String DP	Medium
4	Edit Distance	String DP	Hard
5	Partition Equal Subset Sum	0/1 Knapsack	Medium
6	Maximal Square	Grid DP	Medium
7	Burst Balloons	Interval DP	Hard

Phase 5: Advanced Topics

3–4 weeks

H Heap / Priority Queue

THEORY — LEARN FIRST

- Min-heap vs Max-heap
- heapify, push, pop in $O(\log n)$
- Top-K pattern
- Heap for streaming data

PATTERNS

Top-K

Maintain a heap of size K to find K th largest/smallest.

SIGNAL: K th largest, K closest points, top K frequent

Two Heaps

Max-heap for lower half + min-heap for upper half.

SIGNAL: Median of data stream

Merge K Sorted

Push heads of K lists into heap, extract min each time.

SIGNAL: Merge K sorted lists/arrays, smallest range

PROBLEMS — SOLVE IN ORDER

#	Problem	Pattern	Difficulty
1	Kth Largest Element in Array	Min-Heap	Medium
2	K Closest Points to Origin	Max-Heap	Medium
3	Find Median from Data Stream	Two Heaps	Hard
4	Task Scheduler	Greedy + Heap	Medium
5	Merge K Sorted Lists	Heap	Hard

S Binary Search

THEORY — LEARN FIRST

- Left/right boundary templates
- Search space reduction
- Binary search on answer
- Rotated arrays

PATTERNS

Classic Binary Search

Find exact target in sorted array.

SIGNAL: Find position, first/last occurrence

BS on Answer

Binary search over the answer space, validate with helper.

SIGNAL: Minimize maximum, capacity to ship, split array

Search in Rotated / 2D

Determine which half is sorted, search accordingly.

SIGNAL: Rotated sorted array, search in 2D matrix

PROBLEMS — SOLVE IN ORDER

#	Problem	Pattern	Difficulty
1	Binary Search	Classic	Easy
2	Search in Rotated Sorted Array	Rotated BS	Medium
3	Find Minimum in Rotated Array	Rotated BS	Medium
4	Koko Eating Bananas	BS on Answer	Medium
5	Capacity to Ship Packages	BS on Answer	Medium
6	Median of Two Sorted Arrays	Binary Search	Hard

R Tries & Advanced Strings

THEORY — LEARN FIRST

- Trie node structure
- Insert, search, startsWith in $O(L)$
- Space-time tradeoffs vs HashMap

PATTERNS

Prefix Tree

Build trie for $O(L)$ prefix lookups.

SIGNAL: Autocomplete, word search II, longest word

PROBLEMS — SOLVE IN ORDER

#	Problem	Pattern	Difficulty
1	Implement Trie	Trie	Medium
2	Word Search II	Trie + Backtracking	Hard
3	Design Add & Search Words	Trie + DFS	Medium

HOW TO SOLVE ANY NEW PROBLEM

1 Read — don't code yet	Write what the problem is asking in plain English. No code.
2 Identify the SIGNAL	Is it sorted? Asking for a subarray? A pair? Count of occurrences? Shortest path?
3 Map it to a pattern	Which pattern does this signal belong to? Cross-reference this roadmap.
4 Code the pattern template	Not from scratch — from the skeleton you've drilled via practice.
5 Debug on paper first	Trace through a small example by hand before running code.
6 Optimise	Only after a correct brute-force. Ask: can hashing, sorting, or a smarter DS help?